

Reviewer Pack — Minimal Representation Rank of Matroids Associated with Bool...

Entry f2f8ee401363 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 03:47:17 UTC

Minimal Representation Rank of Matroids Associated with Boolean Functions

Entry ID: f2f8ee401363 Verdict: INCONCLUSIVE Recorded: 2026-05-22 03:47:17 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Boolean Function Complexity Theory

Statement:

[‘For any boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, let $M(f)$ be the matroid associated with the indicator vectors of f . Then the minimal rank of $M(f)$ is at least $\Theta(n \log n)$, where $\log n$ denotes the binary logarithm of n .’, ‘This implies that for any boolean function f and a matroid N with rank less than $n \log n$, there exists no representation of f using linear combinations of the indicator vectors of N .’, ‘For all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, if there exists an algorithm A with complexity $C(f)$ such that for every input x , $A(x)$ produces a matroid N with rank less than $n \log n$ and satisfies $f = \sum_{y \in \{0,1\}^n} N(y)$, then $C(f) = \Theta(n \log n)$.’]

Rationale (proposer’s reasoning):

[‘Matroids provide a combinatorial abstraction of linear independence that has been used in various areas of mathematics. Their representation theory offers a rich source of invariants for combinatorial objects, including boolean functions.’, ‘The conjecture proposes a lower bound on the minimal rank of matroids associated with boolean functions, which would be a novel application of algebraic combinatorics to complexity theory.’, ‘If true, this conjecture would provide a direct connection between the representation theory of matroids and the complexity of boolean function computation.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0d13315490ffd239

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated boolean functions f , the minimal rank of $M(f)$ is at least $n \log n$ with 95% confidence interval across 30 seeds, and no algorithm A

can represent f using a matroid N with rank less than $n \log n$ in time complexity better than $\Theta(n \log n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal representation rank" AND "matroids associated with boolean functions" - "Boolean function complexity theory" AND "Algebraic Combinatorics" - "indicator vectors of boolean functions" AND "matroid rank"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(n + 1):
                A[j][k] += factor * A[i][k]

    rank = n
    for i in range(n):
        if abs(A[i][i]) < 1e-9:
            rank -= 1

    return rank
```

```

def matroid_rank(f, n):
    indicator_vectors = [[int(x[i] == j) for i in range(n)] for j in range(2)]
    A = [v + [f(v)] for v in indicator_vectors]
    return gaussian_elimination(A)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = lambda x: sum(x) % 2
        M_f_rank = matroid_rank(f, n)

        if M_f_rank < n * math.log(n):
            return {
                "metric_name": "matroid_rank",
                "metric_value": M_f_rank,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"Function with rank {M_f_rank} < {n * math.log(n)}"
            }

        results.append(M_f_rank)

    mean = sum(results) / len(results)
    std_dev = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5

    return {
        "metric_name": "matroid_rank",
        "metric_value": mean,
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r >= n * math.log(n)) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r < n * math.log(n) for r in results):
        first_failing_seed = seeds[results.index(next(r for r in results if r < n * math.log(n)))]
        print(f"RESULT: FALSIFIED counterexample='rank<{n*math.log(n)}' first_failing_seed={first_failing_seed}")

```

```
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21f48d50.py", line 83, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21f48d50.py", line 54, in run_trial
    M_f_rank = matroid_rank(f, n)
               ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21f48d50.py", line 42, in matroid_rank
    indicator_vectors = [[int(x[i] == j) for i in range(n)] for j in range(2)]
                        ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15900
2	preregistration	ollama_remote	glm4:latest	0	0	12629
3	novelty	ollama_remote	glm4:latest	0	0	8680
4	novelty	ollama_remote	glm4:latest	0	0	8957
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14115
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26167
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12087
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10064
9	judge	ollama_remote	glm4:latest	0	0	64251

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172850 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f2f8ee401363.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f2f8ee401363.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f2f8ee401363.tar.gz (if generated)